DIGITAL FORENSICS WRITEUP

CyberTalents – Err

Récupération d'une image PNG corrompue

Analyse hexadécimale, réparation de l'en-tête et CRC32

Catégorie : Digital Forensics

Plateforme : CyberTalents

Challenge : Err

Auteur : `laaroussidin`

Documentation technique de la résolution du challenge

Table des matières

1 Introduction **2**

2 Environnement et outils utilisés **2**

3 Résolution du challenge **2**

3.1 Préparation et téléchargement 2

3.2 Identification initiale du fichier 3

3.3 Correction de la signature PNG 4

3.4 Correction du type de chunk IHDR 4

3.5 Analyse des dimensions et du CRC32 5

3.6 Recherche des dimensions correctes avec Python 6

3.7 Réparation finale et ouverture de l'image 7

4 Flag obtenu **8**

5 Bilan et conclusion **9**

6 Références **9**

1. Introduction

Ce writeup présente la résolution du challenge **Err** dans la catégorie **Forensics** de la plateforme CyberTalents. Le but est d'analyser un fichier qui ne s'ouvre pas normalement, de comprendre pourquoi il est endommagé, puis de retrouver son contenu.

Pour résoudre ce challenge, nous avons utilisé des commandes Linux et un petit script Python. La démarche consiste à vérifier le type du fichier, ensuite étudié ses premiers octets, et enfin corrigé son en-tête. La dernière étape a permis de retrouver une image contenant le flag.

2. Environnement et outils utilisés

Outil	Utilité dans le challenge
Kali Linux	Environnement de travail et terminal.
`wget`	Télécharger le fichier du challenge.
`file`	Identifier le type d'un fichier.
`xxd`	Afficher les octets du fichier en hexadécimal.
`cp` et `dd`	Copier le fichier et modifier des octets à un emplacement précis.
Python 3, `zlib`, `struct`	Calculer le CRC32 et rechercher les dimensions correctes.
Visionneuse d'images	Ouvrir l'image réparée pour lire le flag.

3. Résolution du challenge

3.1 Préparation et téléchargement

Nous avons commencé par créer un dossier pour organiser les fichiers du challenge. Ensuite, nous avons téléchargé le fichier fourni par CyberTalents.

```
mkdir -p ~/CTF/Err
cd ~/CTF/Err
wget https://dr-hubchallenges.s3.us-west-2.amazonaws.com/Forensics/err
```

La commande `mkdir -p` crée le dossier, `cd` permet d'y entrer, et `wget` télécharge le fichier nommé `err`.

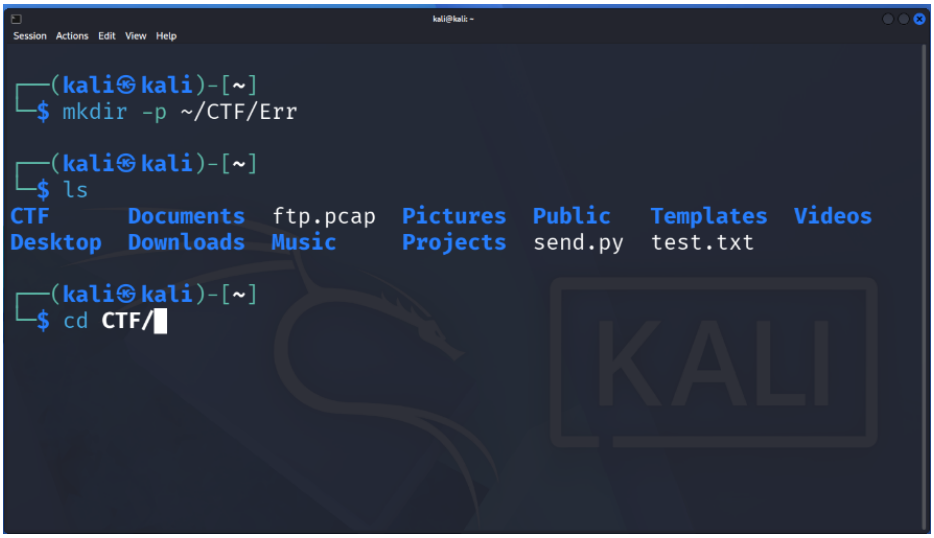


FIGURE 1 – Préparation du dossier de travail sous Kali Linux.



FIGURE 2 – Téléchargement du fichier err depuis le lien du challenge.

3.2 Identification initiale du fichier

Nous avons cherché à identifier le format avec :

```
file err
```

La commande a affiché `err: data`. Cela veut dire qu'elle n'a pas reconnu de format courant à partir des informations présentes dans le fichier. Le nom du fichier n'avait pas non plus d'extension permettant de nous guider.

Pour examiner son contenu, nous avons exécuté :

```
xxd -l 64 err
```

La commande `xxd` affiche les octets en hexadécimal. L'option `-l 64` limite l'affichage aux 64 premiers octets. Nous avons observé le début suivant :

```
00000000: 464c 4147 0d0a 1a0a 0000 000d 464c 4147
```

Les quatre premiers octets `46 4C 41 47` correspondent au texte ASCII `FLAG`. Cependant, les quatre octets suivants `0D 0A 1A 0A` ressemblent à la fin de la signature d'un fichier PNG. Cette observation nous a donné une première piste : **le fichier pouvait être une image PNG dont certains octets avaient été remplacés**.

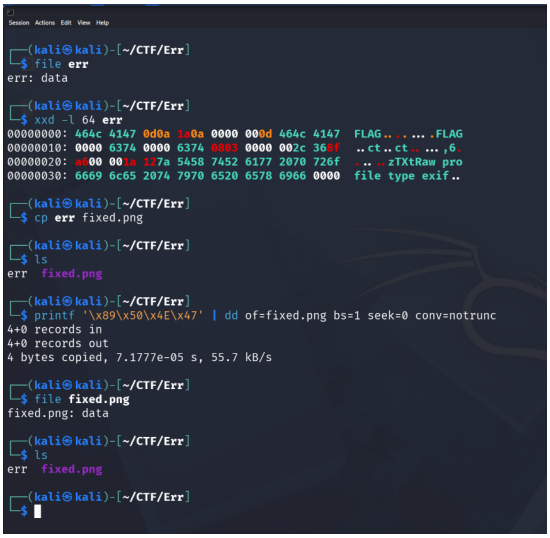


FIGURE 3 – Identification du fichier et inspection des premiers octets.

3.3 Correction de la signature PNG

Une image PNG commence normalement par huit octets précis :

```
89 50 4E 47 0D 0A 1A 0A
```

Dans notre fichier, les quatre premiers octets avaient été remplacés par `46 4C 41 47`. Nous avons créé une copie pour ne pas modifier l'original, puis réparé ces quatre octets :

```
cp err fixed.png
printf '\x89\x50\x4E\x47' | dd of=fixed.png bs=1 seek=0 conv=notrunc
```

`printf` produit les quatre octets attendus. La commande `dd` les écrit dans le fichier : `bs=1` travaille octet par octet, `seek=0` commence au début et `conv=notrunc` évite de raccourcir le fichier.

Après cette correction, `file fixed.png` affichait encore `data`. Nous avons donc compris qu'une autre partie du fichier était endommagée.

3.4 Correction du type de chunk IHDR

Dans un PNG, après la signature viennent quatre octets indiquant la longueur d'un bloc (*chunk*), suivis de quatre octets indiquant son type. Le premier chunk doit s'appeler `IHDR`. Dans notre fichier, nous avions :

```
00 00 00 0D 46 4C 41 47
```

Les octets `00 00 00 0D` indiquent une longueur de 13 octets. Les quatre octets suivants devraient représenter `IHDR`, soit `49 48 44 52`, et non `FLAG`.

Nous avons donc corrigé le type du chunk au décalage 12 :

```
printf '\x49\x48\x44\x52' | dd of=fixed.png bs=1 seek=12 conv=notrunc
file fixed.png
```

Cette fois, `file` a reconnu une image PNG, mais avec des dimensions étonnantes : **25460 x 25460 pixels**. La signature et le type de chunk étaient maintenant reconnus, mais les dimensions semblaient

également corrompues.

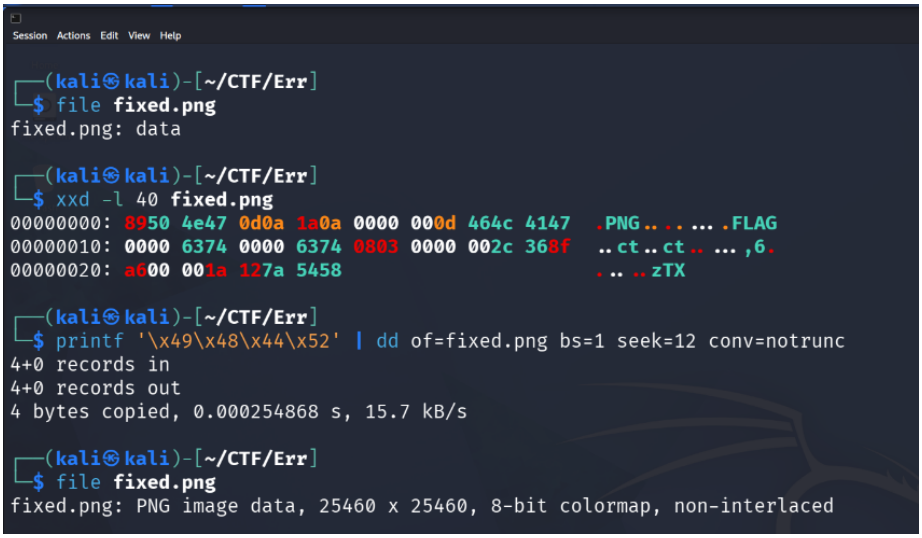


FIGURE 4 – Réparation du type IHDR et identification d'une image aux dimensions anormales.

3.5 Analyse des dimensions et du CRC32

Nous avons utilisé `xxd` pour examiner de plus près le début du fichier :

```
xxd -g 1 -l 64 fixed.png
xxd -g 1 -s 16 -l 8 fixed.png
```

L'option `-g 1` sépare les octets un par un. L'option `-s 16` commence à l'offset 16, là où les dimensions de l'image sont stockées. Nous avons trouvé :

```
Width  : 00 00 63 74
Height : 00 00 63 74
```

Ces valeurs valent `0x6374`, soit **25460** en décimal. Les octets `63 74` correspondent également aux lettres `ct` en ASCII. C'était un indice de modification.

Le chunk `IHDR` contient un CRC32. C'est une valeur de contrôle calculée à partir du type du chunk et de ses données. Elle permet de détecter certaines modifications accidentelles ou volontaires. Dans notre fichier, le CRC32 stocké à la fin du chunk était : `0x2C368FA6`.

Nous avons vérifié si les données actuelles produisaient ce CRC :

```
import zlib

with open("fixed.png", "rb") as f:
    f.seek(12)
    data = f.read(17)  # 4 octets IHDR + 13 octets de donnees

calculated = zlib.crc32(data)
print("Calculated CRC:", hex(calculated))
print("Stored CRC:    ", hex(0x2C368FA6))
print("Match:", calculated == 0x2C368FA6)
```

Le résultat obtenu était :

```
Calculated CRC: 0x102c4054
Stored CRC:     0x2c368fa6
```

```
Match: False
```

Le CRC ne correspondait donc pas aux données actuelles. Nous avons décidé de rechercher des dimensions qui rendent le CRC valide.



FIGURE 5 – Inspection des octets et premier calcul du CRC32.



FIGURE 6 – Comparaison entre CRC calculé et CRC stocké : résultat False.

3.6 Recherche des dimensions correctes avec Python

Nous avons réalisé une recherche automatique des dimensions possibles. Le script teste des largeurs et des hauteurs comprises entre 1 et 2000 pixels. Pour chaque paire, il recalcule le CRC32 du chunk `IHDR`, puis le compare au CRC stocké dans le fichier.

```
import zlib
import struct

target = 0x2C368FA6
properties = bytes.fromhex("0803000000")
```

```
for width in range(1, 2001):
    prefix = b'IHDR' + struct.pack('>I', width)
    for height in range(1, 2001):
        data = prefix + struct.pack('>I', height) + properties
        if zlib.crc32(data) == target:
            print("FOUND!")
            print("Width:", width)
            print("Height:", height)
            raise SystemExit
print("Not found in this range")
```

`struct.pack('>I', valeur)` transforme un entier en quatre octets, dans l'ordre *big-endian* utilisé par PNG. `zlib.crc32` calcule ensuite la valeur de contrôle. Cette technique s'appelle une recherche exhaustive (ou *brute-force*) dans un intervalle limité.

Le programme a trouvé :

```
FOUND!
Width: 1400
Height: 787
```

Ces dimensions sont cohérentes pour une image. Surtout, elles produisent le CRC32 attendu. Cela nous a permis de retenir **1400 x 787** comme dimensions à restaurer.

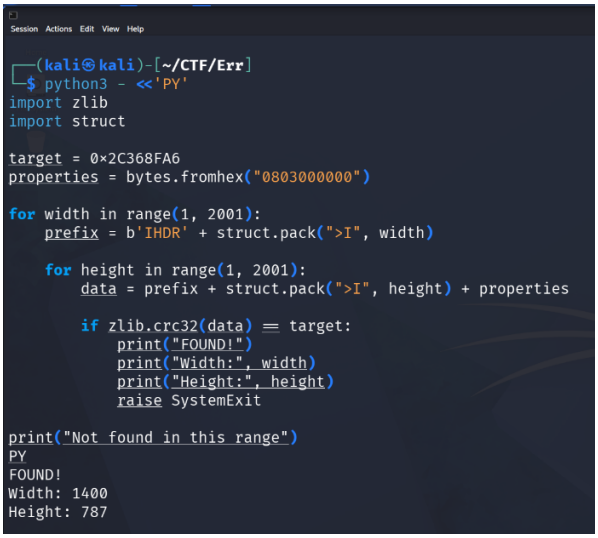


FIGURE 7 – Recherche des dimensions par CRC32 : largeur 1400, hauteur 787.

3.7 Réparation finale et ouverture de l'image

Nous avons créé une nouvelle copie du fichier, puis écrit la largeur et la hauteur retrouvées aux offsets 16 à 23 :

```
cp fixed.png recovered.png
printf '\x00\x00\x05\x78\x00\x00\x03\x13' | \
  dd of=recovered.png bs=1 seek=16 conv=notrunc
file recovered.png
xdg-open recovered.png
```

`00 00 05 78` représente 1400, et `00 00 03 13` représente 787. Après réparation, la commande `file` reconnaissait correctement :

```
recovered.png: PNG image data, 1400 x 787, 8-bit colormap, non-interlaced
```

Nous avons ensuite ouvert le fichier avec `xdg-open`. L'image était lisible et le flag était écrit dessus.



FIGURE 8 – Écriture des dimensions correctes et vérification du fichier recovered.png.

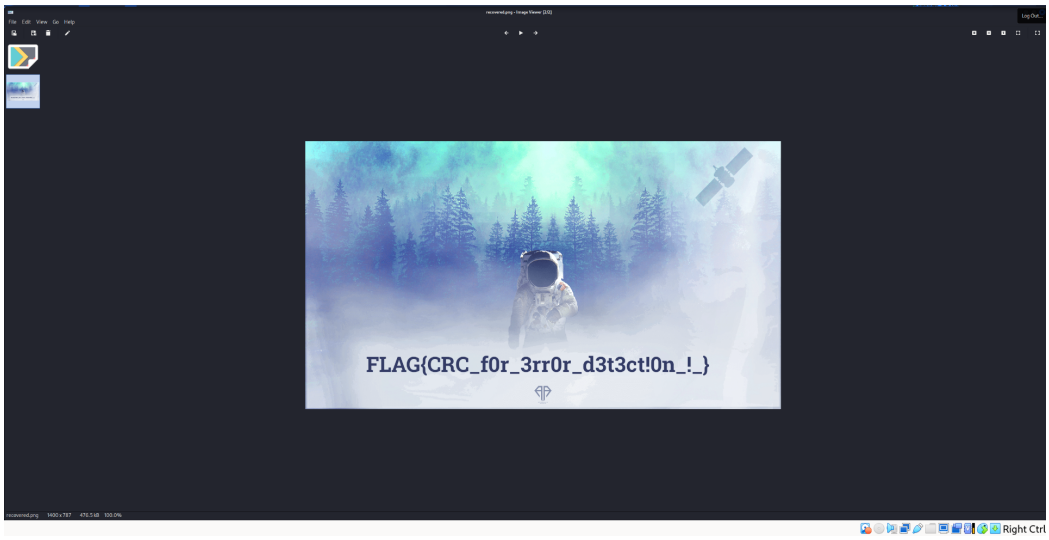


FIGURE 9 – Image récupérée, avec le flag visible.

4. Flag obtenu

Flag du challenge Err :

`FLAG{CRC_f0r_3rr0r_d3t3ct!0n_!_}`

Le flag apparaît directement dans l'image restaurée. Nous avons pu le lire après avoir réparé la signature PNG, le type `IHDR`, puis les dimensions.

5. Bilan et conclusion

Ce challenge nous a permis de découvrir une méthode simple d'analyse forensique : commencer par vérifier le format d'un fichier, examiner ses octets, comparer sa structure avec une signature connue, puis corriger uniquement les parties endommagées. Nous avons aussi appris à utiliser le CRC32 pour vérifier les données d'un fichier PNG et retrouver des dimensions cohérentes.

Le plus important est que nous n'avons pas eu besoin de deviner directement le flag. Nous avons réparé le fichier étape par étape jusqu'à ce que l'image originale redevienne lisible.

6. Références

- Challenge : `https://cybertalents.com/challenges/forensics/err`
- Plateforme : `https://cybertalents.com/`
- Profil CyberTalents : `laaroussidin` (lien public non vérifié).